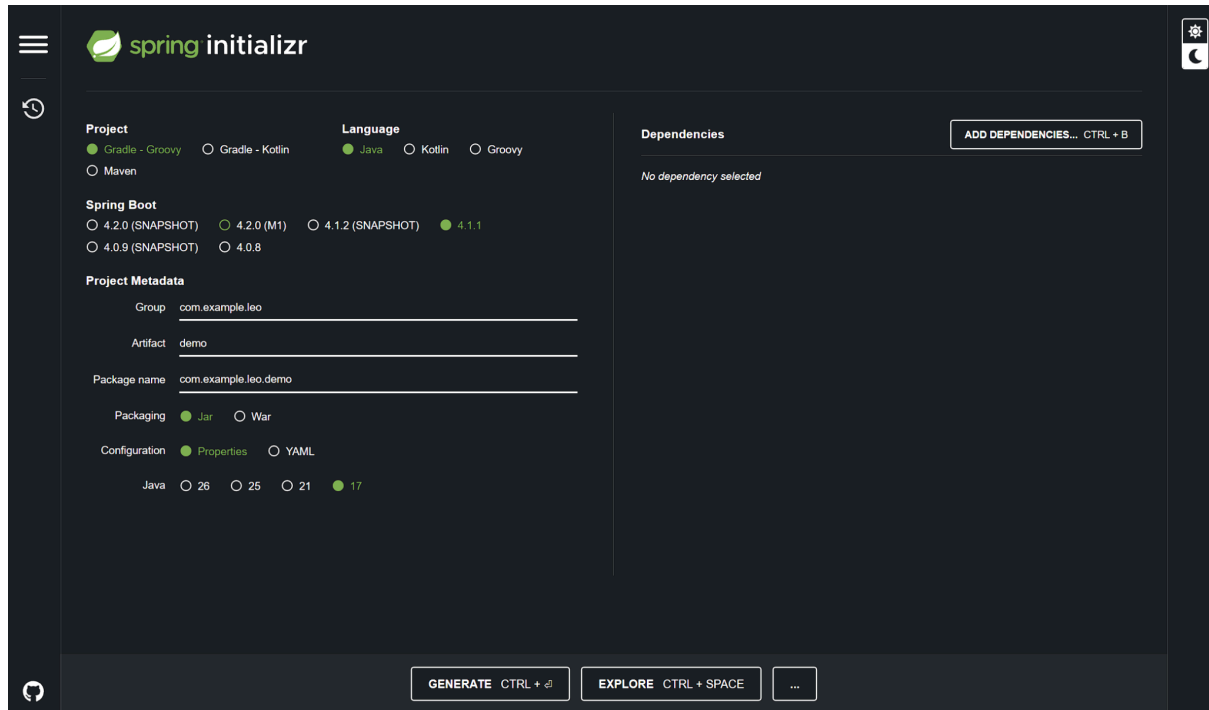


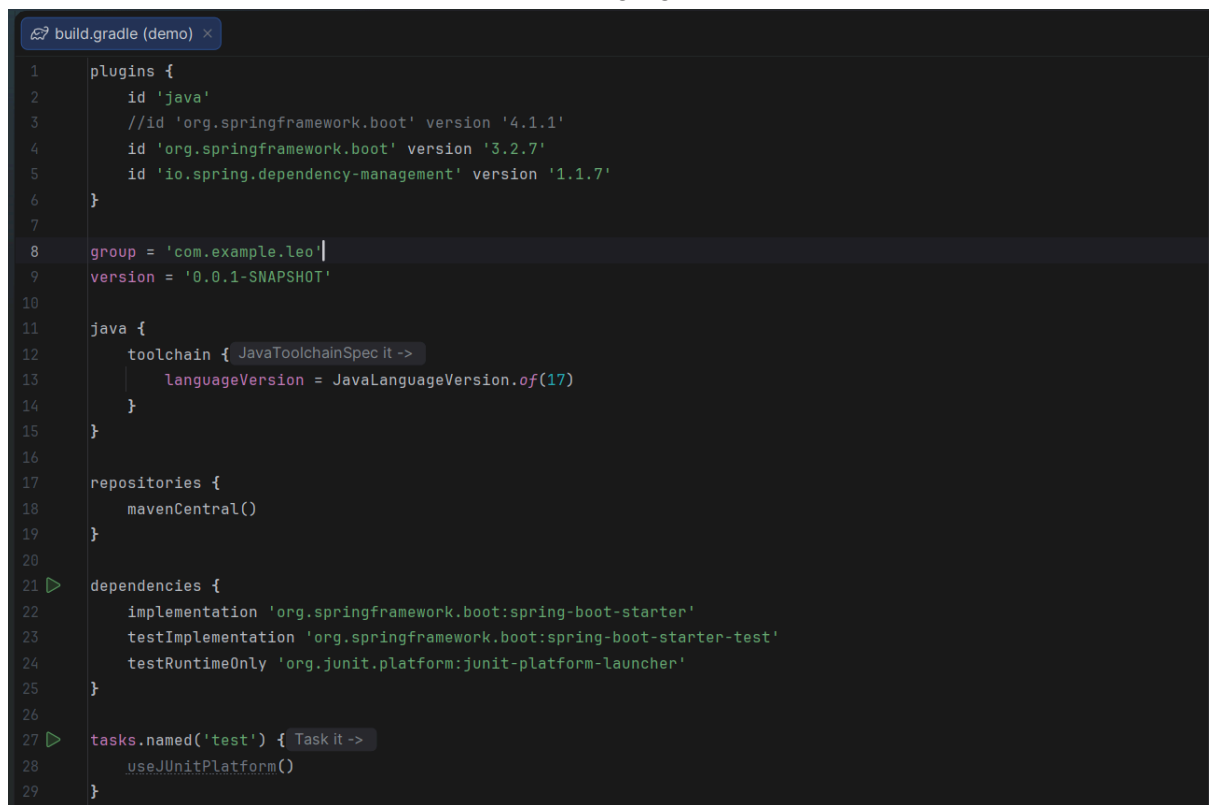
Prueba técnica: Roberto Leonel Perez Ramirez

1. Creo el proyecto en la página de Spring Initializr, pero la versión de Spring Boot 3.2.7 no está disponible, así que uso cualquier otra versión para cambiarla mas tarde, lo primero es generar el proyecto.



The screenshot shows the Spring Initializr web application interface. The 'Project' section has 'Gradle - Groovy' selected. The 'Language' section has 'Java' selected. The 'Spring Boot' section has '4.1.1' selected. The 'Project Metadata' section shows 'Group' as 'com.example.leo', 'Artifact' as 'demo', and 'Package name' as 'com.example.leo.demo'. The 'Packaging' section has 'Jar' selected. The 'Configuration' section has 'Properties' selected. The 'Java' section has '17' selected. The 'Dependencies' section is empty with a button 'ADD DEPENDENCIES... CTRL + B'. At the bottom, there are buttons 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and a menu button '...'.

2. Modifico la versión anterior 4.1.1 para agregar la versión solicitada.

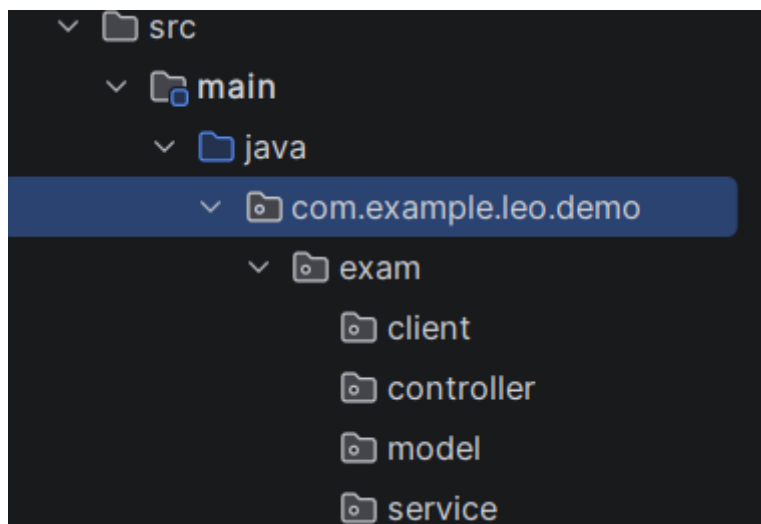


```
1  plugins {
2      id 'java'
3      //id 'org.springframework.boot' version '4.1.1'
4      id 'org.springframework.boot' version '3.2.7'
5      id 'io.spring.dependency-management' version '1.1.7'
6  }
7
8  group = 'com.example.leo'
9  version = '0.0.1-SNAPSHOT'
10
11 java {
12     toolchain { JavaToolchainSpec it ->
13         languageVersion = JavaLanguageVersion.of(17)
14     }
15 }
16
17 repositories {
18     mavenCentral()
19 }
20
21 dependencies {
22     implementation 'org.springframework.boot:spring-boot-starter'
23     testImplementation 'org.springframework.boot:spring-boot-starter-test'
24     testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
25 }
26
27 tasks.named('test') { Task it ->
28     useJUnitPlatform()
29 }
```

3. Ejecuto el comando: `./gradlew bootRun` para que Gradle descargue las dependencias correctas.

[illegible]

4. Organizo el código así: `src\main\java\com\example\leoldemo\exam`
 - controller: APIs REST
 - service: Lógica de negocio
 - client: Conexión a APIs externas
 - model: Objetos de datos



5. Creo las clases: PetService, PetClient, PetController y Pet para el modelo.

```
package com.example.leo.demo.exam.model;

8 usages 1 related problem
public class Pet {
    2 usages
    private Long id;
    2 usages
    private String name;
    2 usages
    private String status;

    no usages
    public Long getId() {
        return id;
    }

    no usages
    public void setId(Long id) {
        this.id = id;
    }

    no usages
    public String getName() {
        return name;
    }

    no usages
    public void setName(String name) {
        this.name = name;
    }

    no usages
    public String getStatus() {
        return status;
    }

    no usages
    public void setStatus(String status) {
        this.status = status;
    }
}
```

```
build.gradle (demo)  gradle-wrapper.properties  Pet.java  PetClient.java x  PetService.java  PetController.java  PetResponse.java

1  package com.example.leo.demo.exam.client;
2
3  import com.example.leo.demo.exam.model.Pet;
4  import org.springframework.stereotype.Component;
5  import org.springframework.web.client.RestTemplate;
6
7  @Component
8  public class PetClient {
9
10     //Usa RestTemplate para realizar llamadas HTTP
11     1 usage
12     private final RestTemplate restTemplate = new RestTemplate();
13     1 usage
14     private final String BASE_URL = "https://petstore.swagger.io/v2/pet/";
15
16     1 usage
17     public Pet getPetById(Long petId) {
18         //Devuelve un objeto Pet con los datos recibidos.
19         return restTemplate.getForObject(BASE_URL + petId, Pet.class);
20     }
21 }
```

```
build.gradle (demo) x gradle-wrapper.properties Pet.java PetClient.java PetService.java x PetController.java PetResponse.java

1 package com.example.leo.demo.exam.service;
2
3 import com.example.leo.demo.exam.client.PetClient;
4 import com.example.leo.demo.exam.model.Pet;
5 import com.example.leo.demo.exam.model.PetResponse;
6 import org.springframework.stereotype.Service;
7
8 @Service
9 public class PetService {
10     2 usages
11     private final PetClient petClient;
12
13     no usages
14     public PetService(PetClient petClient) {
15         this.petClient = petClient;
16     }
17
18     //GET > consume la API externa
19     1 usage
20     public Pet getPet(Long petId) {
21         Pet pet = petClient.getPetById(petId);
22         System.out.println("Pet info: " + pet); // Log en consola
23         return pet;
24     }
25
26     //POST > crea un Pet y devuelve respuesta con metadata
27     1 usage
28     public PetResponse createPet(Pet pet) {
29         System.out.println("Pet creado: " + pet.getName() + " - " + pet.getStatus());
30         return new PetResponse(pet.getName(), pet.getStatus());
31     }
32 }
```

```
build.gradle (demo) gradle-wrapper.properties Pet.java PetClient.java PetService.java PetController.java x PetResponse.java

1 package com.example.leo.demo.exam.controller;
2
3 import com.example.leo.demo.exam.model.Pet;
4 import com.example.leo.demo.exam.model.PetResponse;
5 import com.example.leo.demo.exam.service.PetService;
6 import org.springframework.web.bind.annotation.*;
7
8 no usages
9 @RestController
10 @RequestMapping("/api/pet")
11 public class PetController {
12     3 usages
13     private final PetService petService;
14
15     no usages
16     public PetController(PetService petService) {
17         this.petService = petService;
18     }
19
20     // GET > obtiene un Pet desde la API externa
21     no usages
22     @GetMapping("/{petId}")
23     public Pet getPet(@PathVariable Long petId) {
24         return petService.getPet(petId);
25     }
26
27     // POST > crea un Pet y devuelve respuesta con metadata
28     no usages
29     @PostMapping
30     public PetResponse createPet(@RequestBody Pet pet) {
31         return petService.createPet(pet);
32     }
33 }
```

6. Se crea la clase: **PetResponse** que en lugar de devolver directamente el mismo objeto pet que recibe en el POST, **PetResponse** agrega información extra que hace más útil la respuesta.

```
build.gradle (demo)  gradle-wrapper.properties  Pet.java  PetClient.java  PetService.java  PetController.java  PetResponse.java x
1 package com.example.leo.demo.exam.model;
2
3 import java.time.LocalDateTime;
4 import java.util.UUID;
5
6 public class PetResponse {
7     2 usages
8     private String transactionId;
9     2 usages
10    private LocalDateTime dateCreated;
11    2 usages
12    private String name;
13    2 usages
14    private String status;
15
16    1 usage
17    public PetResponse(String name, String status) {
18        this.transactionId = UUID.randomUUID().toString();
19        this.dateCreated = LocalDateTime.now();
20        this.name = name;
21        this.status = status;
22    }
23
24    // Getters
25    no usages
26    public String getTransactionId() {
27        return transactionId;
28    }
29
30    no usages
31    public LocalDateTime getDateCreated() {
32        return dateCreated;
33    }
34
35    no usages
36    public String getName() {
37        return name;
38    }
39}
```

7. Como salen errores, corrijo mi archivo `gradle-wrapper.properties` para usar **Gradle 8.8**

```
build.gradle (demo)  gradle-wrapper.properties x  Pet.java  PetClient.java  PetService.java  PetController.java
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-9.7.1-bin.zip
networkTimeout=10000
retries=0
retryBackOffMs=500
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

```
build.gradle (demo)  gradle-wrapper.properties x  Pet.java  PetClient.java  PetService.java  PetController.java
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.8-bin.zip
networkTimeout=10000
retries=0
retryBackOffMs=500
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

8. Actualizo el wrapper para que descargue la nueva versión modificada para que sea compatible: `./gradlew wrapper --gradle-version 8.8`

```
Terminal Local x + -
PS C:\Users\leora\Documents\Projects\demo> ./gradlew wrapper --gradle-version 8.8
Starting a Gradle Daemon, 1 incompatible and 2 stopped Daemons could not be reused, use --status for details

BUILD SUCCESSFUL in 10s
1 actionable task: 1 executed
```

9. Limpio y reconstruyo el proyecto: `./gradlew clean build`

```
PS C:\Users\leora\Documents\Projects\demo> ./gradlew clean build
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended

BUILD SUCCESSFUL in 11s
8 actionable tasks: 8 executed
```

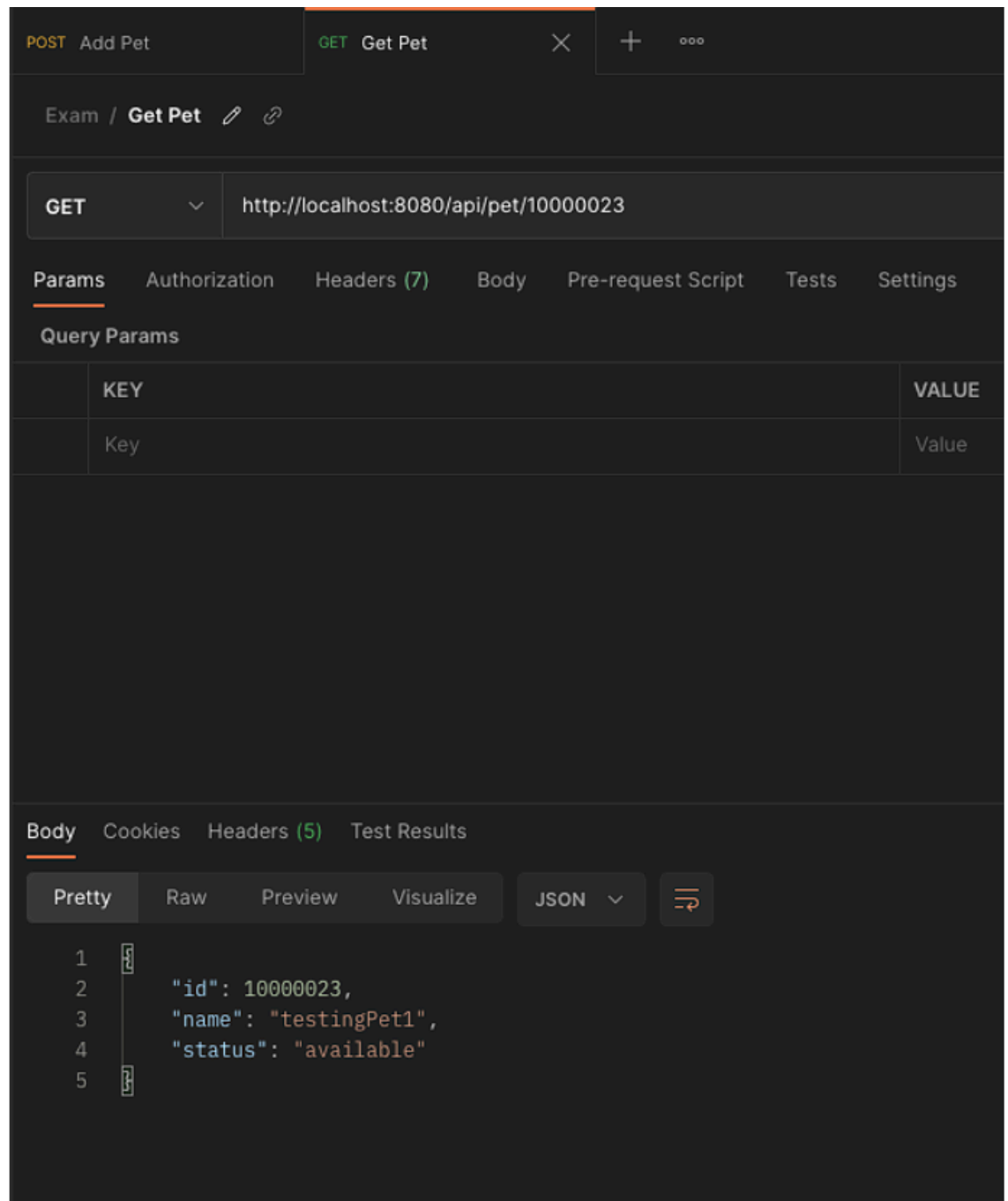
10. Pruebo levantar la aplicación: ./gradlew bootRun

[illegible]

11. Pruebo la API de tipo POST para verificar el funcionamiento

The screenshot displays the Swagger UI interface for a REST API. At the top, there are tabs for Overview, POST http://localhost:8080/, and GET http://localhost:8080/. The main header shows the URL http://localhost:8080/api/pet and a POST method. Below this, there are tabs for Docs, Params, Authorization, Headers, Body, Scripts, and Settings. The Body tab is selected, showing the request body in raw format: {"id": 10000023, "name": "testingPet1", "status": "available"}. The response is shown as 200 OK with a JSON body: {"transactionId": "41cf2642-a55b-49c2-a075-3b53252388c6", "dateCreated": "2026-09-15T09:43:58.4176846", "name": "testingPet1", "status": "available"}. The interface also includes a Save button, a Share button, and a Cookies button.

12. Al probar la API de tipo GET no obtuve respuesta como venia en el documento:

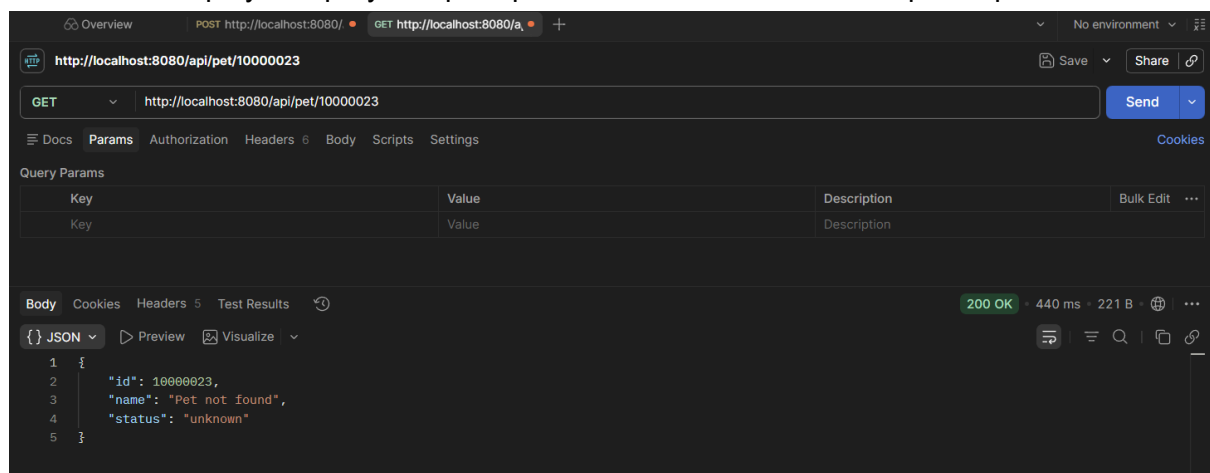


13. Modifique mi clase PetClient para agregar un manejador de errores para ver si el ID era valido o no pues no obtenia respuesta.

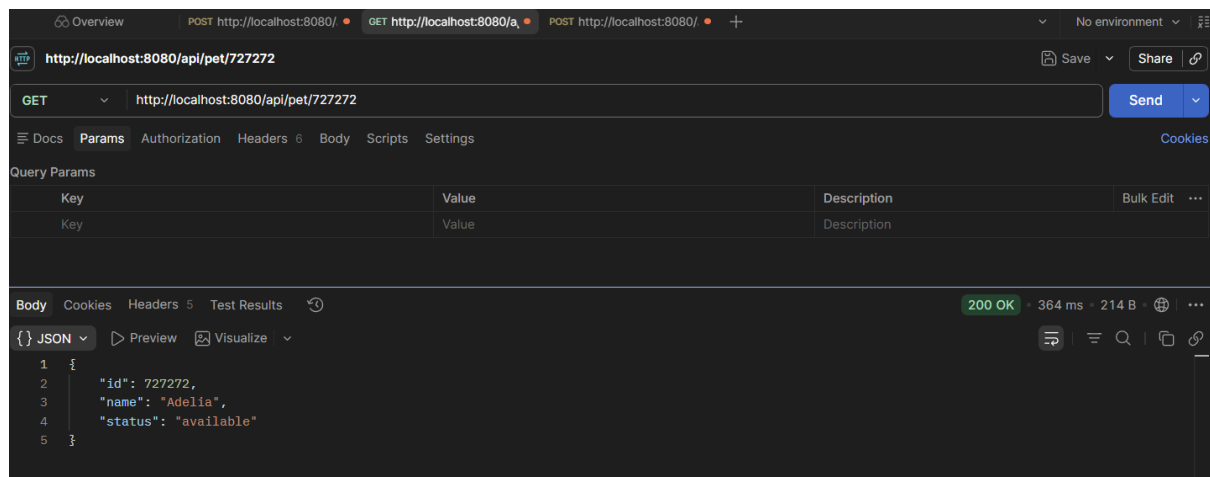
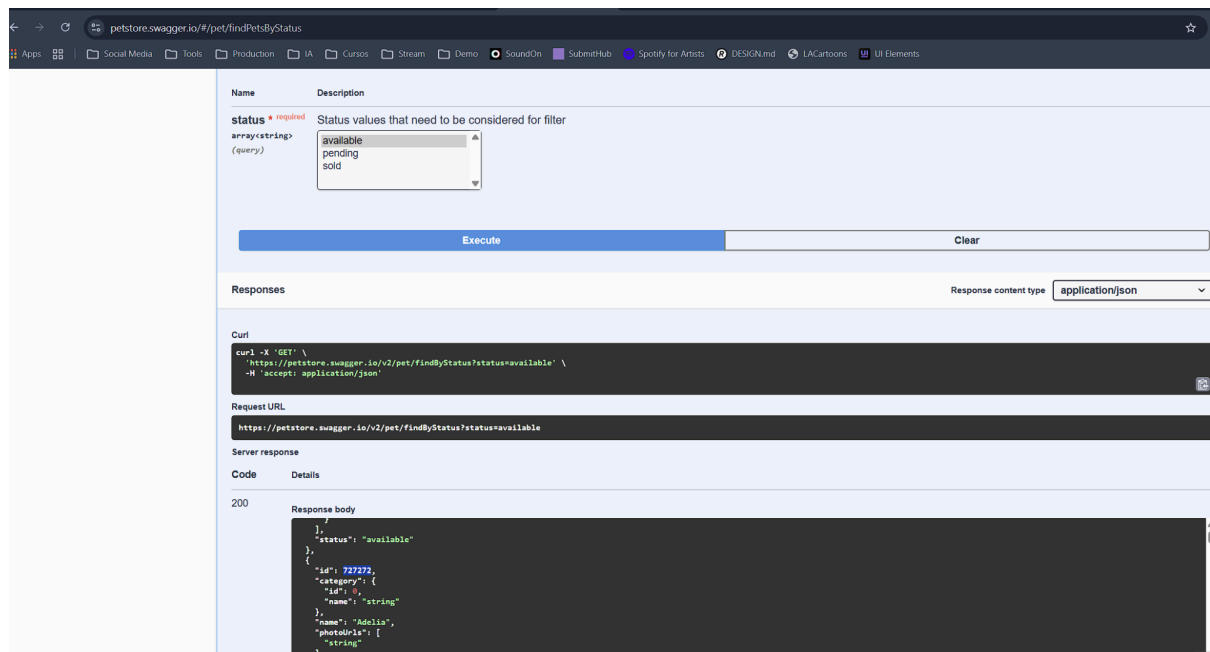
```
build.gradle (demo)  gradle-wrapper.properties  Pet.java  PetClient.java x  PetService.java  PetController.java  PetResponse.java

1 package com.example.leo.demo.exam.client;
2
3 import com.example.leo.demo.exam.model.Pet;
4 import org.springframework.stereotype.Component;
5 import org.springframework.web.client.RestTemplate;
6
7 @Component
8 public class PetClient {
9
10     1 usage
11     private final RestTemplate restTemplate = new RestTemplate();
12
13     2 //usa RestTemplate para realizar llamadas HTTP
14     1 usage
15     private final String BASE_URL = "https://petstore.swagger.io/v2/pet/";
16
17     1 usage
18     public Pet getPetById(Long petId) {
19         try {
20             //devuelve un objeto Pet con los datos recibidos.
21             return restTemplate.getForObject(BASE_URL + petId, Pet.class);
22         } catch (Exception e) {
23             //manejo de errores: si la API falla o el ID no existe
24             System.out.println("Error al consumir API externa: " + e.getMessage());
25             Pet fallback = new Pet();
26             fallback.setId(petId);
27             fallback.setName("Pet not found");
28             fallback.setStatus("unknown");
29             return fallback;
30         }
31     }
32 }
```

15. Volvi a a deployar el proyecto para probar la funcionalidad de la Api de tipo GET:



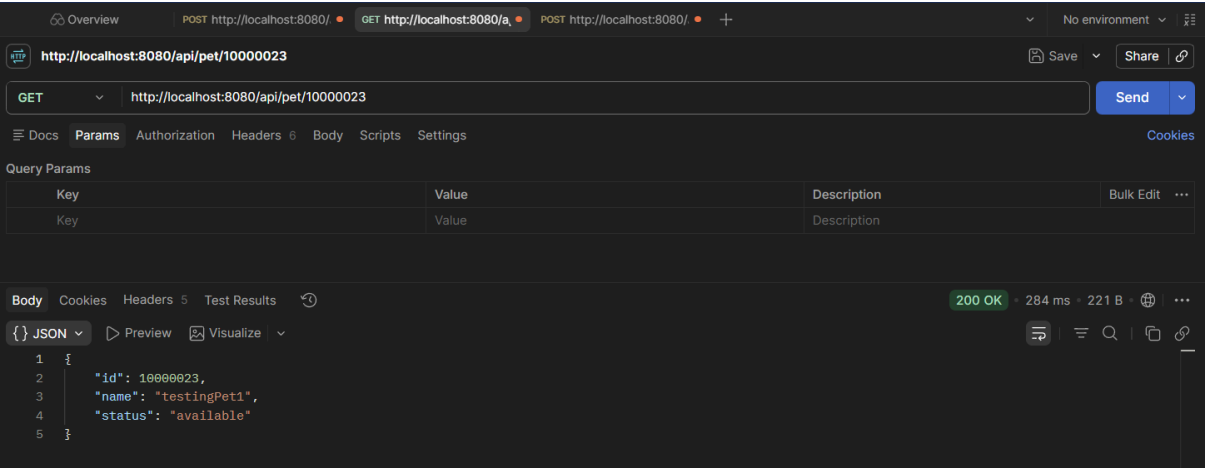
16. Al consultar el API externa de swagger si encuentro resultados por la tanto trato de realizar la insercion POST para obtener un PET que yo envie.



17. Modifico las clases: PetClient y PetService, para apuntar a los servicios externos.

```
build.gradle (demo) x  gradle-wrapper.properties  Pet.java  PetClient.java x  PetService.java  PetController.java  PetResponse.java

1 package com.example.leo.demo.exam.client;
2
3 import com.example.leo.demo.exam.model.Pet;
4 import org.springframework.stereotype.Component;
5 import org.springframework.web.client.RestTemplate;
6
7 @Component
8 public class PetClient {
9
10     2 usages
11     private final RestTemplate restTemplate = new RestTemplate();
12
13     1 usage
14     private final String BASE_URL = "https://petstore.swagger.io/v2/pet/";
15
16
17     // GET: consulta un Pet por ID
18     1 usage
19     public Pet getPById(Long petId) {
20         try {
21             return restTemplate.getForObject(BASE_URL + petId, Pet.class);
22         } catch (Exception e) {
23             System.out.println("Error al consumir API externa: " + e.getMessage());
24             Pet fallback = new Pet();
25             fallback.setId(petId);
26             fallback.setName("Pet not found");
27             fallback.setStatus("unknown");
28             return fallback;
29         }
30     }
31
32
33     // POST: crea un Pet en la API externa
34     no usages
35     public Pet createPet(Pet pet) {
36         try {
37             return restTemplate.postForObject("https://petstore.swagger.io/v2/pet", pet, Pet.class);
38         } catch (Exception e) {
39             System.out.println("Error al crear Pet en API externa: " + e.getMessage());
40             return null;
41         }
42     }
43 }
44 }
```

19. Creo mi propio registro y pruebas para comprobar funcionalidad:

